

AhorraGo Master Report

Historial acumulado de fases técnicas del proyecto.

Fase 1

Fecha: 2026-05-31

Objetivo

Corregir problemas críticos y de alto impacto inicial sin romper el MVP actual, evitando refactor masivo, cambios de arquitectura o migraciones de base de datos.

Cambios Aplicados

1. Seguridad: API Key de Jumbo

- Se eliminó la API key hardcodeada de los scrapers de Jumbo.
- Los scrapers ahora leen `JUMBO\_API\_KEY` desde variables de entorno.
- Si falta la variable, el scraper falla con un mensaje claro sin exponer secretos.
- Se creó `.env.example` con la variable requerida.
- Se actualizó `.gitignore` para excluir `.env` y `.env.\*`, manteniendo `.env.example` versionable.

Archivos:

- `app/scrapper\_jumbo\_real.py`
- `app/scrapper\_jumbo\_api.py`
- `.env.example`
- `.gitignore`

2. Búsquedas con LIMIT y OFFSET

- Se agregaron parámetros `limit` y `offset` a endpoints de búsqueda.
- Default: `limit=50`.
- Máximo permitido: `limit=100`.
- Se evitó carga ilimitada en las búsquedas principales de productos.
- El frontend ahora consulta con `limit=50&offset=0`.

Archivos:

- `app/main.py`
- `app/services.py`
- `frontend/index.html`

3. URLs Correctamente Escapadas

- Se creó helper centralizado para generar URLs de búsqueda con ``urllib.parse.urlencode``.
- Se reemplazaron URLs generadas con ``replace(" ", "%20")``.
- Las URLs ahora soportan tildes, ``ñ`` y caracteres especiales.

Archivos:

- ``app/url_utils.py``
- ``app/convertir_lider.py``
- ``app/convertir_jumbo.py``
- ``app/convertir_unimarc.py``
- ``app/generar_catalogo.py``
- ``app/services.py``

4. Índices Básicos en SQLite

- Se reforzó el script idempotente de índices.
- Se agregaron índices seguros para:
 - ``productos.nombre``
 - ``productos.producto_base``
 - ``precios.producto_id``
 - ``precios.supermercado_id``
- El script usa ``CREATE INDEX IF NOT EXISTS``, por lo que no borra ni reconstruye datos.

Archivo:

- ``app/scripts/agregar_indices.py``

5. Lock Real para Actualización de Productos

- Se reemplazó el lock manual por ``filelock``.
- Se evita que dos ejecuciones del pipeline corran al mismo tiempo.
- Si ya existe una actualización en curso, el proceso sale con mensaje claro.

Archivos:

- ``app/actualizar_productos.py``
- ``requirements.txt``

6. Tests Iniciales

Se creó el primer set mínimo de tests para:

- Normalización de texto.
- Equivalencia entre ``Leche Soprole 1L`` y ``Leche Soprole 1000ml``.
- URLs con tildes y ``ñ``.

- Límites de búsqueda.
- Cálculo básico de ahorro.

Archivo:

- `tests/test\_phase1.py`

7. Documento para Agentes

Se creó `AGENTS.md` con:

- Stack detectado.
- Reglas de trabajo para Codex.
- Comandos de ejecución.
- Comandos de testing.
- Reglas de seguridad.
- Definición de terminado.

Archivo:

- `AGENTS.md`

Dependencias Agregadas

En `requirements.txt`:

```
filelock>=3.13.0
pytest>=8.0.0
```

Verificación Ejecutada

Comandos ejecutados:

```
venv\Scripts\python.exe -m pytest tests/test_phase1.py -q
venv\Scripts\python.exe -m compileall app tests
venv\Scripts\python.exe -m app.scripts.agregar_indices
venv\Scripts\python.exe -c "from app.main import app; print(app.title)"
```

Resultados:

- Tests: `5 passed`.
- Compilación Python: OK.
- Import de FastAPI app: OK.
- Índices SQLite aplicados correctamente.
- Verificación de `JUMBO\_API\_KEY` faltante: muestra error claro.

Archivos Modificados o Creados

Modificados:

- `.gitignore`

- `app/actualizar\_productos.py`
- `app/convertir\_jumbo.py`
- `app/convertir\_lider.py`
- `app/convertir\_unimarc.py`
- `app/generar\_catalogo.py`
- `app/importar\_csv.py`
- `app/main.py`
- `app/scrapper\_jumbo\_api.py`
- `app/scrapper\_jumbo\_real.py`
- `app/scripts/agregar\_indices.py`
- `app/services.py`
- `frontend/index.html`
- `requirements.txt`

Creados:

- `.env.example`
- `AGENTS.md`
- `app/url\_utils.py`
- `tests/test\_phase1.py`
- `CAMBIOS\_FASE\_1.md`

Riesgos Pendientes

- Aún existen `.all()` en endpoints de diagnóstico, estado de datos y lógica interna no crítica. No se tocaron para evitar ampliar demasiado esta fase.
- El pipeline completo de scrapers no fue ejecutado para evitar scraping externo y cambios masivos de datos.
- Hay un warning de Pydantic por uso de `@validator`; conviene migrarlo a `@field\_validator` en una fase posterior.
- SQLite se mantiene como base actual, según la restricción de no migrar todavía.

Siguiente Fase Recomendada

1. Optimizar endpoints pesados como `/diagnostico/calidad` y `/estado-datos`.
2. Agregar tests de integración con SQLite temporal.
3. Migrar validadores Pydantic a estilo v2.
4. Revisar límites y paginación en comparador y diagnósticos.
5. Revisar manejo de configuración con carga explícita de `.env` si el flujo local lo necesita.

Fase 2

Fecha: 2026-05-31

Objetivo

Reducir deuda técnica sin romper el MVP, enfocándose en `main.py`, normalización duplicada, matching de productos, endpoints pesados y tests de integración.

Cambios Aplicados

1. Normalización compartida

- Se creó `app/normalizacion.py`.
- Se movieron funciones de normalización y comparación de claves desde `main.py`.
- `importar\_csv.py` ahora usa `generar\_producto\_base` desde el módulo compartido.
- Se mantuvo compatibilidad con imports existentes usados por tests.
- Se normalizaron equivalencias de formato:
 - `1L` y `1000ml`.
 - `500g` y `0.5kg`.
 - `pack 6` y `6 unidades`.

Archivos:

- `app/normalizacion.py`
- `app/main.py`
- `app/importar\_csv.py`

2. Matching de productos

- Se creó `app/matching.py`.
- Se movió `candidato\_compatible` y sus reglas relacionadas a un módulo dedicado.
- `main.py` ahora importa esa lógica en vez de contenerla directamente.
- Se agregaron tests unitarios de equivalencias.

Archivos:

- `app/matching.py`
- `app/main.py`
- `tests/test\_matching.py`

3. Reducción segura de `main.py`

- Se retiró lógica de normalización y matching desde `main.py`.
- Los endpoints actuales se mantienen con las mismas rutas y estructura de respuesta.
- No se reestructuró la API ni se cambió la UX.

Archivo:

- `app/main.py`

4. Optimización de endpoints pesados

- `/diagnostico/calidad`` ahora consulta columnas necesarias y usa ``yield_per(500)`` en vez de cargar objetos completos con ``all()``.
- `/estado-datos`` ahora calcula conteos por supermercado con ``GROUP BY`` y conteos directos.
- Se mantuvo la forma de respuesta esperada por el frontend.

Archivo:

- ``app/main.py``

5. Tests de integración mínimos

- Se agregó SQLite temporal en memoria para pruebas de API.
- Se prueba búsqueda básica con datos mínimos.
- Se prueba límite de resultados.
- Se prueba estado de datos con conteos básicos.

Archivo:

- ``tests/test_integration.py``

6. Pydantic v2

- Se migró ``@validator`` a ``@field_validator``.
- El warning de Pydantic v2 desaparece en la suite de tests.

Archivo:

- ``app/schemas.py``

7. Dependencias

Se agregó ``httpx``, requerido por ``fastapi.testclient.TestClient`` para los tests de integración.

Archivo:

- ``requirements.txt``

Comandos Ejecutados

```
$env:PATH = "$PWD\venv\Scripts;$env:PATH"; python -m pytest -q
$env:PATH = "$PWD\venv\Scripts;$env:PATH"; python -m compileall app tests
$env:PATH = "$PWD\venv\Scripts;$env:PATH"; python -c "from app.main import app; print(app.title)"
```

Resultados

- Tests: ``11 passed``.
- Compilación: OK.
- Import de FastAPI app: OK, salida ``FastAPI``.

Archivos Modificados o Creados

Modificados:

- ``app/importar_csv.py``
- ``app/main.py``
- ``app/schemas.py``
- ``requirements.txt``

Creados:

- ``app/normalizacion.py``
- ``app/matching.py``
- ``tests/test_matching.py``
- ``tests/test_integration.py``
- ``CAMBIOS_FASE_2.md``

Riesgos Pendientes

- Todavía quedan ``all()`` en rutas livianas (``categorias``, ``subcategorias``) y en lógica interna acotada; no se tocaron para mantener esta fase pequeña.
- ``main.py`` aún contiene lógica de búsqueda y resumen de compra que podría separarse en una fase posterior.
- El matching sigue siendo heurístico; requiere más fixtures reales para cubrir casos de marcas, sabores, formatos familiares y productos con nombres ambiguos.
- No se ejecutó pipeline completo de scrapers para evitar cambios masivos de datos.

Siguiente Fase Recomendada

1. Extraer búsqueda de productos desde ``main.py`` a un service dedicado.
2. Agregar fixtures realistas para matching por supermercado.
3. Cubrir ``/productos/resumen-compra`` con tests de integración.
4. Revisar endpoints internos con ``all()`` restantes y paginar si empiezan a crecer.
5. Evaluar carga explícita de ``env`` local para mejorar experiencia del scraper sin exponer secretos.

Fase 3

Fecha: 2026-05-31

Objetivo

Aumentar la precisión del matching de productos y del comparador sin cambiar la arquitectura principal, sin migrar la base de datos y sin modificar el frontend.

Cambios Aplicados

1. Normalización avanzada

Se amplió ``app/normalizacion.py`` para reconocer equivalencias de formato:

- Volumen:
 - ``1L = 1000ml``
 - ``1.5L = 1500ml``
 - ``2L = 2000ml``
- Peso:
 - ``1kg = 1000g``
 - ``0.5kg = 500g``
 - ``250g = 0.25kg``
- Cantidad:
 - ``pack 6``
 - ``6 un``
 - ``6 unidades``
 - ``x6``

Esto reduce falsos negativos cuando distintos supermercados publican el mismo formato con unidades diferentes.

2. Extractor de atributos estructurados

Se agregó ``extraer_atributos(nombre_producto)``, que devuelve señales estructuradas para el matching:

```
{
  "marca": "soprole",
  "volumen": 1000,
  "peso": None,
  "unidad": "ml",
  "cantidad": None,
  "categoria": "leche",
  "sabores": [],
  "tokens": [...]
}
```

Ejemplo cubierto por tests:

```
extraer_atributos("Leche Soprole Entera 1L")
```

Devuelve marca ``soprole``, volumen ``1000``, unidad ``ml`` y categoría ``leche``.

3. Matching Score 0-100

Se creó ``matching_score(producto, candidato)`` en ``app/matching.py``.

El score combina:

- Marca.
- Volumen o peso normalizado.

- Cantidad o pack.
- Categoría/familia.
- `producto\_base`.
- Intersección de palabras clave.
- Similitud textual con RapidFuzz.

`candidato\_compatible` ahora mantiene filtros duros para evitar falsos positivos y usa el score como señal final.

4. RapidFuzz

Se agregó `rapidfuzz` a `requirements.txt` y se usa `fuzz.token\_set\_ratio` para evitar depender de comparaciones exactas de texto.

Esto mejora casos como:

- `Coca Cola` vs `Coca-Cola`.
- Reordenamientos de palabras.
- Diferencias menores en nombres publicados por supermercado.

5. Casos problemáticos cubiertos

Se agregaron tests para:

- `Leche Soprole 1L` vs `Leche Soprole 1000ml`.
- `Coca Cola 1.5L` vs `Coca Cola 1500ml`.
- `Arroz Tucapel 1kg` vs `Arroz Tucapel 1000g`.
- `Pack 6 Coca Cola` vs `Coca Cola x6`.
- Yogurt de sabores distintos, que no debe coincidir.

6. Fixtures reales

Se creó `tests/fixtures/productos\_reales.json` con ejemplos representativos de Jumbo, Lider y Unimarc.

Incluye pares equivalentes y un caso negativo de yogurt con sabores distintos.

Archivos Modificados o Creados

Modificados:

- `app/normalizacion.py`
- `app/matching.py`
- `requirements.txt`

Creados:

- `tests/test\_matching\_score.py`

- `tests/test\_fixtures\_matching.py`
- `tests/fixtures/productos\_reales.json`
- `CAMBIOS\_FASE\_3.md`

Tests Agregados

- Tests unitarios para extractor de atributos.
- Tests unitarios para score de matching.
- Tests con fixtures reales.
- Tests negativos para evitar coincidencias incorrectas por sabor.

Comandos Ejecutados

```
$env:PATH = "$PWD\env\Scripts;$env:PATH"; python -m pytest -q
$env:PATH = "$PWD\env\Scripts;$env:PATH"; python -m compileall app tests
$env:PATH = "$PWD\env\Scripts;$env:PATH"; python -c "from app.main import app; print(app.title)"
```

Resultados

- Tests: `19 passed`.
- Compilación: OK.
- Import de FastAPI app: OK, salida `FastAPI`.

Precisión Mejorada

La precisión mejora porque el matching ya no depende solo de texto o formato literal:

- Las unidades se comparan en una representación común.
- Los packs se detectan como cantidad.
- La marca se normaliza antes de comparar.
- RapidFuzz tolera diferencias de orden, guiones y texto adicional.
- Los sabores conocidos actúan como freno para evitar falsos positivos.

Riesgos Pendientes

- El sistema de sabores aún es una lista curada y puede necesitar ampliación con datos reales.
- El umbral de `matching\_score >= 68` está calibrado con tests iniciales, pero debe validarse con más productos reales.
- Algunas categorías complejas, como detergentes, papel higiénico y promociones multipack, podrían requerir reglas específicas.
- No se ejecutó el pipeline completo de scrapers para evitar cambios masivos de datos.

Próximos Pasos Recomendados

1. Medir precisión con una muestra real etiquetada manualmente.

2. Agregar fixtures para detergentes, papel higiénico, aceites y cafés.
3. Registrar `matching\_score` en diagnósticos internos para revisar falsos positivos.
4. Ajustar umbrales por categoría si aparecen patrones distintos.
5. Crear un reporte de productos agrupados por `producto\_base` para detectar errores de equivalencia.

Fase 4

Fecha: 2026-05-31

Objetivos

Mejorar la calidad de datos y aumentar equivalencias reales entre supermercados antes de crear usuarios, manteniendo todo en modo seguro:

- Sin usuarios.
- Sin login.
- Sin cambios de frontend.
- Sin migración de base de datos.
- Sin scraping masivo.
- Sin modificar datos reales.
- Con respaldo previo.
- Con simulación por defecto.

Métricas Iniciales

- Productos: 31.124
- Precios: 31.139
- Producto_base conflictivos: 1.601
- Productos duplicados: 244
- Productos sospechosos: 3
- Equivalencias detectadas: 2.538
- Sin equivalencia: 79,94%

Métricas Finales

La fase fue de diagnóstico y simulación, por lo que no cambió la base real.

- Total productos analizados: 31.124
- Producto_base únicos: 27.418
- Grupos con equivalencia: 2.538
- Porcentaje sin equivalencia recalculado por diagnóstico: 81,03%

- Producto_base conflictivos clasificados: 1.601
- Score promedio muestreado: 70,91
- Productos evaluados en simulación: 31.124
- Producto_base que cambiarían en simulación: 29.542
- Porcentaje de cambio simulado: 94,92%
- Grupos con equivalencia propuestos: 915
- Riesgo bajo: 2.001 productos
- Riesgo medio: 126 productos
- Riesgo alto de falso positivo: 2 productos

Conflictos Encontrados

Clasificaciones más frecuentes:

- Formato y peso distinto: 856 grupos.
- Formato y volumen distinto: 659 grupos.
- Marca distinta: 21 grupos.
- Error de normalización con volumen distinto: 21 grupos.
- Sabor distinto con volumen distinto: 7 grupos.

Categorías con Peor Matching

- Frutas y Verduras: 7,41% equivalencia.
- Congelados: 7,70% equivalencia.
- Panadería: 13,12% equivalencia.
- Desayuno y Snacks: 14,24% equivalencia.
- Carnes y Pescados: 14,90% equivalencia.

Categorías con Mejor Matching

- Mascotas: 57,18% equivalencia.
- Bebidas: 28,01% equivalencia.
- Limpieza: 27,50% equivalencia.
- Bebé: 24,47% equivalencia.
- Higiene Personal: 23,35% equivalencia.

Archivos Creados

- `app/matching\_diagnostics.py`
- `app/scripts/diagnostico\_matching.py`
- `app/scripts/simular\_reconstruccion\_producto\_base.py`

- `tests/test\_fase4\_diagnostico.py`
- `reports/diagnostico\_matching.md`
- `reports/conflictos\_clasificados.csv`
- `reports/equivalencias\_por\_categoria.csv`
- `reports/grupos\_sospechosos.csv`
- `reports/simulacion\_producto\_base.csv`
- `reports/simulacion\_producto\_base.md`
- `reports/FASE\_4\_REPORTE.md`
- `reports/FASE\_4\_REPORTE.pdf`
- `reports/AHORRAGO\_MASTER\_REPORT.pdf`
- `CAMBIOS\_FASE\_4.md`

Archivos Modificados

- `app/main.py`
- `app/normalizacion.py`
- `tests/fixtures/productos\_reales.json`
- `tests/test\_integration.py`

Scripts Creados

- `python -m app.scripts.diagnostico\_matching`
- `python -m app.scripts.simular\_reconstruccion\_producto\_base`

Tests Agregados

- Clasificación de conflictos.
- Métricas por categoría.
- Simulación de reconstrucción sin modificar la base.
- Nuevas reglas de normalización.
- Falsos positivos y falsos negativos.
- Endpoint `/diagnostico/matching`.

Seguridad de Datos

Se creó respaldo automático antes de la fase:

- `backups/supercheck\_20260531\_213244.db`

La simulación no modificó datos reales.

Riesgos Pendientes

- El 94,92% de cambios propuestos en simulación indica que no debe aplicarse automáticamente.
- La simulación propone menos grupos con equivalencia que el estado actual; sirve para diagnóstico, no para migración directa.
- El mayor problema actual está en grupos `producto\_base` demasiado amplios con formatos/pesos/volúmenes incompatibles.
- Se requiere revisión manual o reglas por categoría antes de cualquier limpieza real.

Recomendaciones

1. No aplicar reconstrucción automática todavía.
2. Priorizar corrección de `producto\_base` en bebidas, limpieza, mascotas y lácteos.
3. Crear una muestra etiquetada manualmente de 300 a 500 productos.
4. Ajustar reglas por categoría con métricas de precisión y recall.
5. Mantener `/diagnostico/matching` como endpoint interno antes de cambios de datos.

Fase 5A

Fecha: 2026-05-31

Objetivo

Mejorar equivalencias únicamente en categorías de mejor retorno y menor riesgo:

- Bebidas
- Limpieza
- Higiene Personal
- Bebé
- Mascotas

No se modificaron categorías de alto riesgo como Frutas y Verduras, Carnes y Pescados, Panadería o Congelados.

Modo de Trabajo

La fase se ejecutó completa en ****DRY RUN****:

- No se modificó la base real.
- No se ejecutó scraping.
- No se modificó frontend.
- No se crearon usuarios.

Reglas Implementadas

Bebidas

- Normalización de `1L`, `1000ml`, `1.5L`, `1500ml`, `2L`, `2000ml`.
- Packs `pack 6`, `x6`, `pack 12`, `x12`.
- Diferenciación de retornable/no retornable.
- Diferenciación de zero/sin azúcar/light vs normal.
- Protección contra sabores distintos.

Limpieza

- Normalización de ml, litros, gramos y kilogramos.
- Detección de aroma.
- Detección de concentrado, diluido, antibacterial y tradicional.
- Protección contra Lavanda vs Limón y Antibacterial vs Tradicional.

Higiene Personal

- Detección de shampoo, acondicionador, jabón, desodorante, pasta dental y crema.
- Detección de aroma.
- Detección de género/formato: hombre, mujer, infantil, adulto.
- Protección contra hombre vs mujer e infantil vs adulto.

Bebé

- Detección de etapa.
- Detección de talla.
- Detección de cantidad.
- Protección contra Talla M vs Talla G y Etapa 1 vs Etapa 2.

Mascotas

- Detección de perro/gato.
- Detección de peso.
- Detección de etapa: cachorro, adulto, senior.
- Protección contra cachorro vs adulto y gato vs perro.

Métricas Actuales vs Proyectadas

- Productos evaluados: 10.656
- Productos equivalentes actuales: 2.918
- Productos equivalentes proyectados: 4.477
- Mejora proyectada: 53,43%
- Conflictos actuales en categorías objetivo: 715
- Conflictos reducibles estimados: 540

- Grupos riesgosos excluidos: 639
- Pares riesgosos detectados y no aplicados: 5.480
- Falsos positivos estimados aplicables: 0

Categorías Más Beneficiadas

- Mascotas: +73,28%
- Limpieza: +63,64%
- Bebé: +59,17%
- Higiene Personal: +59,13%
- Bebidas: +40,60%

Archivos Creados

- `app/fase5a\_rules.py`
- `app/scripts/simular\_mejora\_matching\_fase5a.py`
- `tests/test\_fase5a\_rules.py`
- `tests/test\_fase5a\_simulacion.py`
- `reports/fase5a\_simulacion.md`
- `reports/fase5a\_detalle\_categoria.csv`
- `reports/fase5a\_falsos\_positivos.csv`
- `reports/FASE\_5A\_REPORTE.md`
- `reports/FASE\_5A\_REPORTE.pdf`
- `CAMBIOS\_FASE\_5A.md`

Archivos Modificados

- `app/normalizacion.py`
- `tests/fixtures/productos\_reales.json`

Tests

Se agregaron pruebas positivas, negativas y casos límite para:

- Bebidas
- Limpieza
- Higiene Personal
- Bebé
- Mascotas
- Simulación DRY RUN

Resultado final:

- `32 passed`

Riesgos Pendientes

- La simulación excluye 639 grupos riesgosos; esos grupos requieren revisión manual.
- Los 5.480 pares riesgosos detectados no deben aplicarse automáticamente.
- La mejora proyectada depende de aplicar solo grupos seguros.
- Las categorías excluidas siguen pendientes para fases posteriores.

Recomendación Fase 5B

Ejecutar limpieza controlada por categoría, empezando por Mascotas y Limpieza porque muestran mejor retorno y reglas más claras. Mantener DRY RUN, respaldo automático y aplicación real solo para grupos con riesgo bajo y tests específicos.

Fase 5B

Fecha: 2026-05-31

Objetivo

Aplicar mejoras reales y controladas de matching unicamente en las categorías de menor riesgo:

- Limpieza
- Mascotas

La fase no modifíco Bebidas, Bebe, Higiene Personal, Frutas y Verduras, Congelados, Panadería, Carnes y Pescados.

Cambios Aplicados

- Se creó backup validado antes de modificar la base.
- Se seleccionaron solo grupos seguros usando reglas de Fase 5A.
- Se excluyeron grupos riesgosos, ambiguos o con conflictos de formato, aroma, etapa, sabor o especie.
- Se aplicaron cambios por lote sobre producto_base.
- Se generó trazabilidad completa en CSV.
- Se creó script de rollback.
- Se extendió /diagnostico/matching con métricas de equivalencias actuales, equivalencias por categoría, cambios aplicados y fecha de última actualización.

Métricas Antes/Después

- Productos modificados: 502
- Equivalencias en categorías objetivo: 698 -> 924

- Conflictos en categorias objetivo: 234 -> 163

Por categoria:

- Limpieza: 306 productos modificados, equivalencias 451 -> 632, conflictos 166 -> 133.
- Mascotas: 196 productos modificados, equivalencias 247 -> 292, conflictos 68 -> 30.

Seguridad de Datos

Backup pre-aplicacion validado:

- backups/supercheck_pre_fase5b_20260531_215846.db

Rollback disponible:

```
python -m app.scripts.rollback_fase5b
```

CSV de trazabilidad:

- reports/fase5b_cambios.csv

Validacion:

- reports/fase5b_validacion.md

Archivos Creados

- app/fase5b_apply.py
- app/scripts/aplicar_matching_fase5b.py
- app/scripts/rollback_fase5b.py
- tests/test_fase5b_apply.py
- reports/fase5b_cambios.csv
- reports/fase5b_validacion.md
- reports/FASE_5B_REPORTE.md
- reports/FASE_5B_REPORTE.pdf
- CAMBIOS_FASE_5B.md

Archivos Modificados

- app/matching_diagnostics.py
- reports/AHORRAGO_MASTER_REPORT.md
- reports/AHORRAGO_MASTER_REPORT.pdf
- supercheck.db

Tests

Resultado final esperado de validacion:

- 40 passed
- compileall OK
- Import de app.main OK
- Aplicacion Fase 5B idempotente sin cambios pendientes despues de la primera ejecucion real

Riesgos Pendientes

- Bebidas, Bebe e Higiene Personal quedaron fuera de la aplicacion real.
- Los grupos excluidos por seguridad requieren revision manual o reglas adicionales.
- La base SQLite fue modificada de forma controlada; el rollback esta disponible si se necesita revertir.
- Las categorias de alto riesgo siguen pendientes para fases especificas.

Recomendacion Fase 5C

Preparar una aplicacion controlada para Bebidas usando lista blanca, validacion manual por muestra y reglas estrictas para retornable/no retornable, sabor, zero/light/sin azucar y formatos multipack.